

High-Performance Code Optimizations with Accuracy Guarantees

j.w.w. Eva Darulova, Thomas Köehler,
and Neea Rusch

12 June 2026



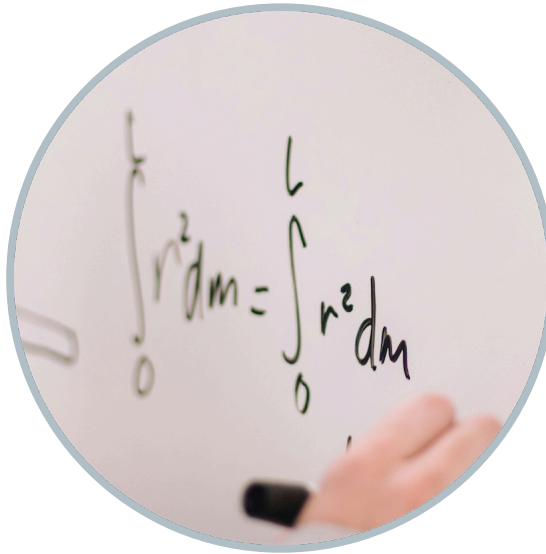
enhance usefulness
or usability



improve performance



ensure correctness
or give guarantees



In instruction level data parallelization:

“SIMD instructions, whose width is a predetermined byte size, can typically process twice as many floats as doubles per cycle. Therefore, performance-sensitive applications are very likely to favor lower-precision alternatives when implementing their algorithms.”

Courbet, C. (2021). NSan: a floating-point numerical sanitizer. *Proceedings of the 30th ACM SIGPLAN International Conference on Compiler Construction*, 83–93. <https://doi.org/10.1145/3446804.3446848>

Project Hepatica

Investigate the tradeoff in **accuracy vs. efficiency** in floating-point computations.

Starting with a program, we generate its semantically equivalent variants.

1. How can we practically and effectively explore the search space of candidate variants?
2. Which variants show favorable or unfavorable accuracy/efficiency characteristics?
3. How do we *actually* measure the differences in the characteristics?

Generally: How to bring accuracy and efficiency under a unified model of reasoning?

Project Hepatica

Investigate the tradeoff in **accuracy vs. efficiency** in floating-point computations.

In Hepatica, we combine and build on two separate lines of prior work:

- **RISE & Shine**: a compiler ecosystem for high performance code optimization.
- **Daisy**: an analysis framework for round-off error measurement and accuracy reasoning.

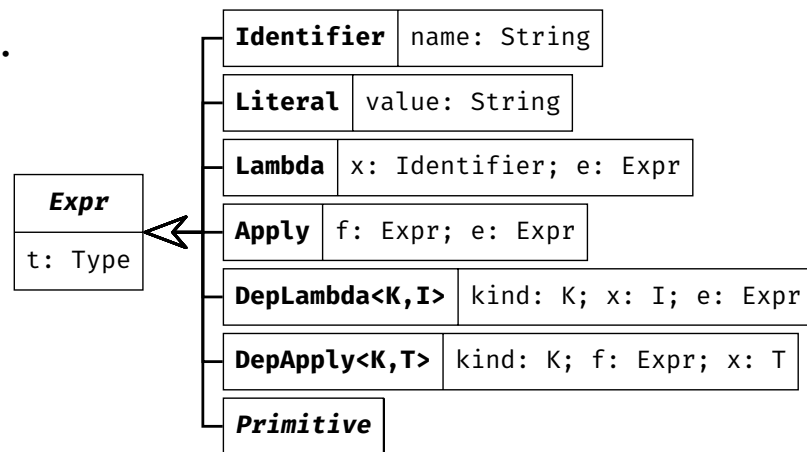
The combination strategy has many advantages, but it is “not easy.”

Importantly, it allow us to answer questions that were previously out-of-reach.

Input programming language: RISE

RISE is a functional pattern-based data-parallel language.

- Based on well known data-parallel patterns, like map/reduce (excl. recursion).
- Programs convey functional behavior.
- Scheduling strategy is defined separately.
- Powerful enough to express many practical scientific computational kernels.



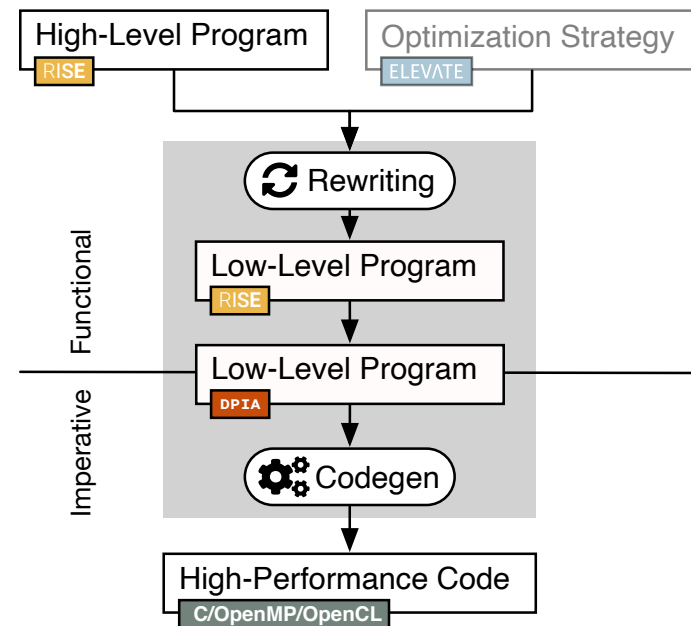
expressions grammar (simplified)²

² [Figure 2] of Steuwer, M., Koehler, T., Köpcke, B., & Pizzuti, F. (2022). *RISE & Shine: Language-Oriented Compiler Design*. <https://doi.org/10.48550/arXiv.2201.03611>

The Shine compiler

A parallelizing source-to-source compiler from RISE to high-performance code.³

1. Rewrites a high-level program into a low-level program.
2. Low-level program encodes all implementation and optimization decisions explicitly.
3. Codegen translates the low-level program into high-performance code.



³ Steuwer, M., Remmelg, T., & Dubach, C. (2017). LIFT: A functional data-parallel IR for high-performance GPU code generation. *2017 IEEE/ACM CGO*, 74–85. <https://doi.org/10.1109/CGO.2017.7863730>

Shine compilation stages

1. High-level RISE →

```
def highLevelMatmul =
  depFun((n: Nat, m: Nat, o: Nat) =>
    fun(n`. `o`. `f32)(A =>
      fun(m`. `o`. `f32)(B =>
        A |> map(fun(rowOfA =>
          B |> map(fun(rowOfB =>
            zip(rowOfA)(rowOfB) |>
              map(fun(x =>
                fst(x) * snd(x)))
            |> reduce(add)(l(0.0f))
          )))))))
```

2. Low-level RISE →

```
def lowLevelMatmul =
  depFun((n: Nat, m: Nat, o: Nat) =>
    fun(n`. `o`. `f32)(A =>
      fun(m`. `o`. `f32)(B =>
        A |> mapPar(fun(rowOfA =>
          B |> mapSeq(fun(rowOfB =>
            zip(rowOfA)(rowOfB) |>
              map(fun(x =>
                fst(x) * snd(x)))
            |> reduceSeq(add)(l(0.0f))
          )))))))
```

3. OpenMP Code

```
#include <stdint.h>

void foo(
  float* output, int n, int m,
  int o, float* A, float* B)
{
  #pragma omp parallel for
  for (int i=0; i<n; i=1+i) {
    for (int j=0; j<m; j=1+j) {
      float acc = 0.0f;
      for (int k=0; k<o; k=1+k) {
        acc += A[k+i*o] * B[k+j*o];
      }
      output[j+i*m] = acc;
    } } }
```

Matrix multiplication

Accuracy analysis challenges

- Static analysis must deduce sound but informative error bounds.
- The ideal, infinite-precision error bounds are often not available for comparison.
- Analyses tend to consider straight-line programs, reducing analysis scalability.
- Handling non-straight-line programs is an open challenge.⁴

In response, several complementary analysis theories and implementations exist.

Overall tooling landscape is fragmented.

⁴ Isychev, A., & Darulova, E. (2023). Scaling up Roundoff Analysis of Functional Data Structure Programs. In M. V. Hermenegildo & J. F. Morales (Eds.), *Static Analysis: Static Analysis*. https://doi.org/10.1007/978-3-031-44245-2_17

The Daisy Framework

An accuracy-focused modular framework for finite-precision computations⁵.

- Provides a unified and extensible environment for verification and development of program analyses and optimizations specialized in accuracy.
- Built-in features: multiple number system support, sound round-off error analysis, rewriting optimizations, transcendental function support, SMT solver interfaces, etc.

⁵ Darulova, E., Izycheva, A., Nasir, F., Ritter, F., Becker, H., & Bastian, R. (2018). Daisy - Framework for Analysis and Optimization of Numerical Programs (Tool Paper). In *Tools and Algorithms for the Construction and Analysis of Systems* (pp. 270–287). Springer. https://doi.org/10.1007/978-3-319-89960-2_15

Program analysis à la Daisy

```
import daisy.lang._
import Real._

object Object {
  def function1(x: Real): Real = {
    require(0 <= x && x <= 1)

    val z = x + x
    z * x

  } ensuring(res => res +/- 1e-5)
}
```

The input language is a subset of Scala.

Ranges of input variables must be defined.

The require clause includes the specification of input values.

The ensuring clause contains an assertion of the absolute round-off error, expressed as magnitude e on variable x .

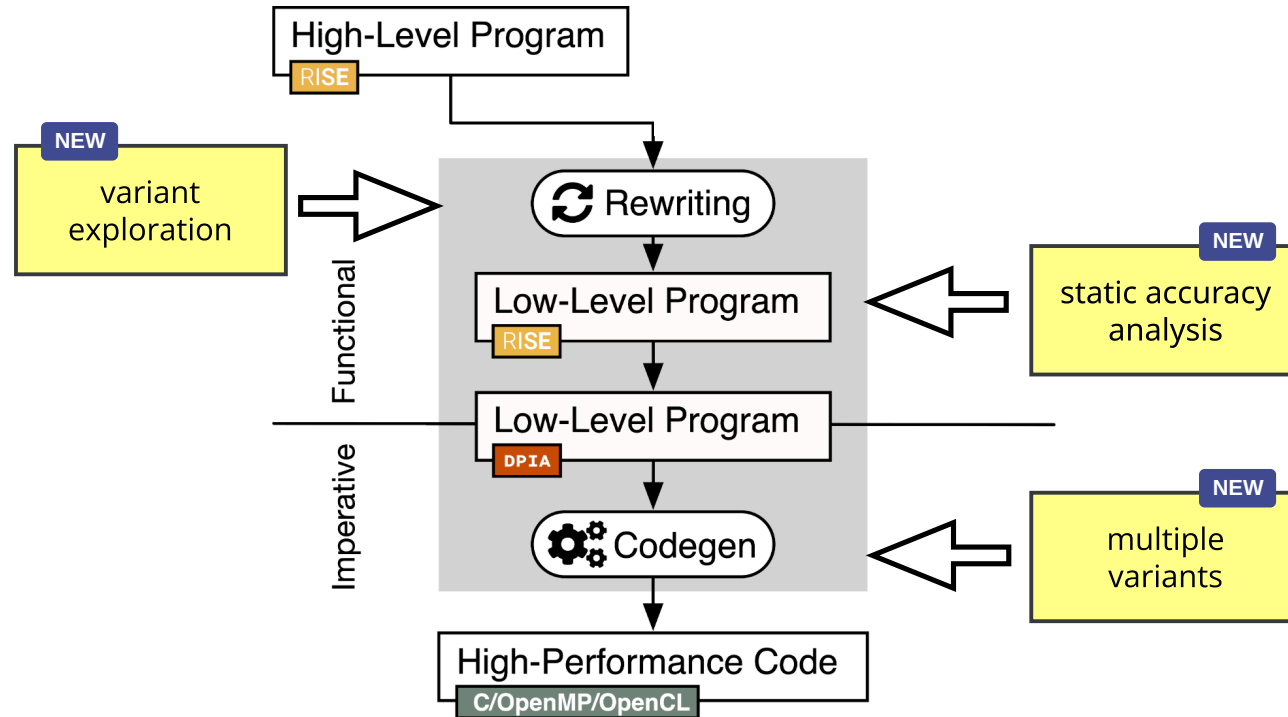
Project Hepatica

In Hepatica, we merge RISE & Shine and Daisy to enable concurrent reasoning about accuracy and efficiency.

Some requirements:

- Need a common model of reasoning: shared programming language (RISE)
- RISE programs must be augmented with specifications
- The accuracy analysis must be implemented on RISE programs
- An evaluation strategy to measure accuracy and efficiency

Augmented Shine compilation workflow



Evaluation Strategy

Our evaluation strategy considers two evaluation phases.

Static analysis to provide compile-time feedback about accuracy.

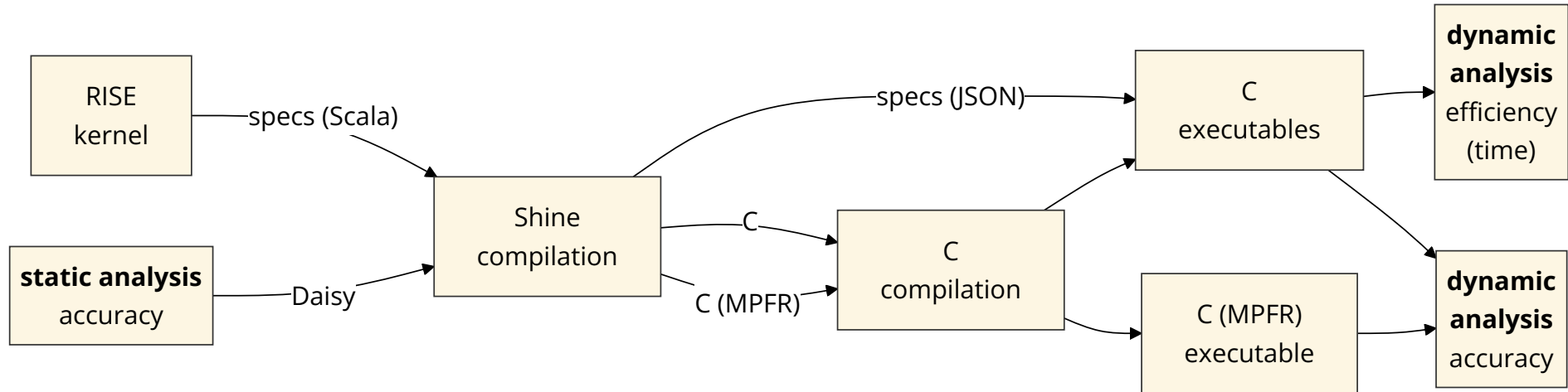
- Must be fast enough to analyze many rewriting variants.

Dynamic analysis of executable code.

- Codegen now produces multiple C program variants and a precise MPFR variant.
- The C program is measured for running time and round-off error.
- The MPFR variant gives the precise ideal round-off error for comparison.

Our benchmark suite comes prior RISE & Shine and Daisy literature.

The full evaluation workflow

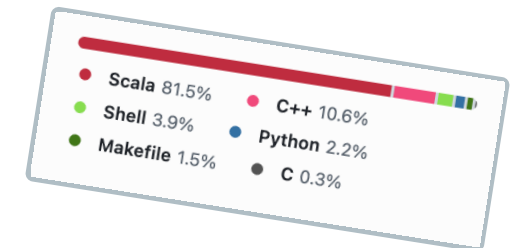
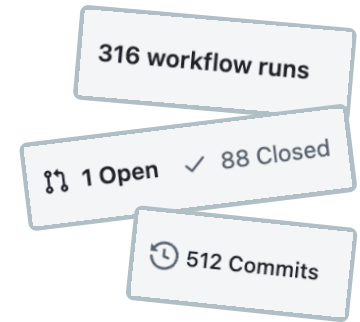


(There are 4 compilers hiding in this picture)

Some resolved and unresolved challenges

- Finding the right representation for program specs.
- Optimization erasing let bindings and producing suboptimal C code.
- Finding the appropriate sampling strategy, edge cases cause abnormal errors.
- Missing a & in C code can slowdown execution 500x.
- Value errors in some benchmarks that have never been evaluated dynamically.
- Translating benchmarks between languages; some are not easily convertible.
- In dynamic executions, how to determine convergence criterion for accuracy?
- If an abnormal error is observed, what input values caused it?
- How many bits of precision do we need for MPFR executions?

...and many more



Hepatica project status

Our complete workflow is operational, from start to finish.

Our current evaluation includes 27 benchmarks (3 input sizes/each).

We are actively resolving outstanding challenges:

- Static analysis covers 14 of 27 benchmarks.
- Depending on configuration, we get hundreds of executable C program variants.
- Dynamic execution to measure time, for 600 variants, takes about 29h.
- Dynamic accuracy analysis requires executing fewer variants, but takes longer.
- Dynamic accuracy experiments will may take multiple days or weeks.

